

MULUNGUSHI UNIVERSITY

Pursuing the frontiers of knowledge

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & IT

ICS341: Cybersecurity Principles

Lab4: *Wireshark TLS 1.3 Packet Analysis & Keycloak OIDC Integration*
Cryptographic Token Verification & Protocol Security Dissection

Lecturer: Kabaso B Winston (Mr.)

Due date: 27th October, 2026 @ 23:59 pm

1 INTRODUCTION & THEORETICAL CONTEXT

Modern transport security and federated identity management constitute the foundation of zero-trust enterprise security. Transport Layer Security (TLS) version 1.3 completely overhauls transport-layer confidentiality and integrity by streamlining key exchange latency to a single round-trip time (1-RTT), mandating Ephemeral Diffie-Hellman (ECDHE) key exchange to guarantee perfect forward secrecy, and encrypting handshake metadata—including server identity certificates.

Simultaneously, enterprise access control has shifted away from monolithic basic authentication toward federated identity protocols. OpenID Connect (OIDC)—built on top of the OAuth 2.0 authorization framework—utilizes JSON Web Tokens (JWT) signed via public-key cryptography (RS256) to communicate identity state statelessly across distributed web services.

This lab challenges Year 3 undergraduate students across Cybersecurity, Computer Science, and Information Technology to analyze, deploy, configure, and cryptographically validate these protocols in a controlled laboratory environment.

2. Laboratory Aim & Objectives

The primary aim of this laboratory exercise is to equip students with hands-on technical proficiency in inspecting secure transport protocols and configuring identity federation infrastructure.

Specific Learning Objectives:

1. **Protocol Dissection:** Capture and inspect a full TLS 1.3 handshake at the packet level using Wireshark, identifying key exchange extensions, cipher suite selections, and encrypted handshake fields.
2. **Session Decryption:** Export browser TLS session secrets (SSLKEYLOGFILE) to decrypt TLS 1.3 application data streams and inspect plaintext HTTP payloads.
3. **Identity Infrastructure:** Deploy an enterprise-grade Keycloak Identity Provider (IdP) container using Docker Engine and Docker Compose
4. **Identity Federation:** Configure OpenID Connect (OIDC) realms, clients, and user credentials to execute an Authorization Code Grant Flow.
5. **Cryptographic Engineering:** Develop a Python verification service to fetch public key sets (JWKS), parse JSON Web Tokens (JWT), and mathematically validate RS256 digital signatures.
6. **Vulnerability Analysis:** Demonstrate and defend against common JWT signature validation flaws, including the alg: 'none' attack vector and RS256-to-HS256 algorithm confusion.

3. Hardware & Software Requirements

Execute this lab on existing personal virtual workstation computers. No physical bare-metal installation is required.

Component/Tool	Specification/Version	Role & Functional Necessity in Lab
Host Operating System	Windows 10/11, macOS 12+, or Linux	Provides host environment for Docker containers, Wireshark, Python, and browsers.
Docker Engine & Compose	Docker Desktop 4.20+ / Docker 24.0+	Hosts containerized Keycloak IdP without requiring native Java/Wildfly database setups.
Wireshark Packet Analyzer	Wireshark v4.0.0 or newer	Captures raw interface packets to inspect TLS 1.3 handshake frames and perform payload decryption.
Modern Web Browser	Google Chrome or Mozilla Firefox	Interacts with Keycloak Admin Console, triggers OIDC flows, and logs master key secrets.
Python Development Env	Python 3.10+ (PyCharm / VS Code)	Used to build the OIDC backend token verification script and test JWT exploit payloads
CLI Security Utilities	OpenSSL 3.0+, cURL, Git	OpenSSL inspects certs/keys; cURL queries OIDC endpoints directly; Git manages lab files

4. Environment Setup Procedure (Step-by-Step)

Step 1: Wireshark Installation & Master Key Export Configuration

1. **Installation:** Download and install Wireshark (v4.0+) ensuring NPCAP (Windows) or Dumpcap (macOS/Linux) packet capture drivers are enabled.
2. **Secret Logging Setup:** Configure environment variable to log TLS master key secrets to a text file:
   - Windows (PowerShell as Administrator):
     ```
     [System.Environment]::SetEnvironmentVariable('SSLKEYLOGFILE', "$env:USERPROFILE\sslkeys.log", 'User')
     ```
   - macOS / Linux (Bash/Zsh Terminal):
     ```
     export SSLKEYLOGFILE="$HOME/sslkeys.log"
     echo 'export SSLKEYLOGFILE="$HOME/sslkeys.log"' >> ~/.bashrc
     ```
3. **Verification:** Restart your browser and verify that opening an HTTPS site populates entries inside the sslkeys.log file.

Step 2: Keycloak Container Deployment via Docker

2.1 Compose File Creation: Create a project directory named lab4-keycloak and add the following docker-compose.yml file:

```
version: '3.8'
services:
  keycloak:
    image: quay.io/keycloak/keycloak:24.0.0
    container_name: keycloak_lab4
    environment:
      - KEYCLOAK_ADMIN=admin
      - KEYCLOAK_ADMIN_PASSWORD=admin
    command:
      - start-dev
    ports:
      - "8080:8080"
```

2.2 Launch Container: Launch the container: docker compose up -d

2.3 Verify Access: Navigate to http://localhost:8080 and log into Administration Console using admin / admin.

Step 3: Python Environment & Cryptographic Library Setup

3.1 Virtual Environment: Initialize virtual environment and install dependencies:

```
python3 -m venv venv
source venv/bin/activate  # On Windows: .\venv\Scripts\activate
pip install pyjwt cryptography requests
```

5. Lab Execution Instructions

Part A: Wireshark TLS 1.3 Packet Analysis & Decryption

Task A1: Capture Handshake. Launch Wireshark and start capturing packets on your active network interface (Wi-Fi or Ethernet). Set capture filter to tcp port 443.

Task A2: Trigger Session. Open browser and navigate to https://www.cloudflare.com. Stop capture once fully loaded.

Task A3: Inspect ClientHello. Apply display filter tls and locate the ClientHello frame. Inspect Extension: supported_groups and Extension: key_share.

Task A4: Inspect ServerHello. Locate ServerHello. Identify the chosen cipher suite (e.g., TLS_AES_256_GCM_SHA384) and Server Key Share curve.

Task A5: Decrypt Payload. In Wireshark, go to Edit -> Preferences -> Protocols -> TLS. Set (Pre)-Master-Secret log filename to path of your sslkeys.log file. Observe that encrypted application data frames immediately resolve to HTTP/2 decrypted traffic.

Part B: Keycloak OIDC Configuration & JWT Validation

Task B1: Realm Setup. Log into Keycloak Admin Console. Create a new Realm named CybersecurityLab.

Task B2: Client Setup. Create a Client named python-oidc-app. Enable Client Authentication, set Valid Redirect URIs to http://localhost:5000/callback.

Task B3: User Provisioning. Create a User named student1 with password Password123! under Users menu.

Task B4: Token Retrieval. Execute authorization code request via browser to retrieve code, then exchange code for tokens via cURL:

```
curl -d "client_id=python-oidc-app" \
     -d "client_secret=<YOUR_CLIENT_SECRET>" \
     -d "grant_type=authorization_code" \
     -d "code=<AUTH_CODE>" \
     -d "redirect_uri=http://localhost:5000/callback" \
     http://localhost:8080/realms/CybersecurityLab/protocol/openid-connect/token
```

Task B5: Python Verifier script. Develop Python backend script lab4_verifier.py that dynamically fetches JWKS public keys from http://localhost:8080/realms/CybersecurityLab/protocol/openid-connect/certs and validates signature of returned RS256 Access Token.

Part C: JWT Vulnerability Exploitation Challenge

Task C1: Alg 'None' Attack. Modify your Python verifier to simulate an insecure decoder that accepts alg: 'none'. Construct a forged JWT payload with role: admin, strip the signature, and verify if the flawed logic accepts it.

Task C2: Algorithm Confusion & Hardening. Simulate RS256-to-HS256 algorithm confusion by passing Keycloak's public key as an HMAC secret to sign a forged admin token. Implement strict algorithm whitelisting in your Python verifier to block both attacks.

6. Lab Report Deliverables & Defense Questions

Your report must comprise:

1. Cover Page (e.g., Group Name, Project Title, Lab Title, Team Lead).
2. Introduction:
3. Main Body:
4. Conclusion:
5. References:

NB: Each group must elect the team lead, who shall lead the team during this lab activity as this is a rotating role.

The team must compile a comprehensive technical report with well captioned images or snippets on each task, and prepare for group oral defense based on the following multi-tiered questions and other related:

Part A Deliverables (TLS 1.3 Packet Analysis):

Q1.1 Handshake Capture: Provide screenshot of Wireshark capture showing ClientHello, ServerHello, and Encrypted Extensions. Annotate exact frame numbers.

Q1.2 Cipher Analysis: Which cryptographic curves were offered in ClientHello key_share? Why are static RSA cipher suites absent in TLS 1.3?

Q1.3 Privacy Analysis: Why are Server Certificate messages encrypted in TLS 1.3? Contrast this privacy feature with TLS 1.2 plain text transmission.

Q1.4 Decryption & 0-RTT: Provide screenshot of decrypted HTTP/2 payload after loading sslkeys.log. Explain the mechanics and replay attack risks of TLS 1.3 0-RTT session resumption.

Part B Deliverables (Keycloak OIDC & JWT Validation):

Q2.1 Token Anatomy: Provide decoded JSON payload for ID Token, Access Token, and Refresh Token issued by Keycloak. Explain purpose of each.

Q2.2 Verifier Code: Paste Python code used to fetch JWKS and cryptographically verify RS256 signature locally using public key components (N and e).

Q2.3 Exploit Analysis: Explain the exact security failure of alg: 'none' and algorithm confusion attacks. Show Python code implementing algorithm whitelisting.

Q2.4 Storage Security: Compare storing JWTs in localStorage vs HttpOnly; Secure; SameSite=Strict cookies regarding XSS and CSRF attack surfaces.

Part C: Group Collaboration & Team Role Synthesis:

Q3.1 IT Role: IT / System Admin members explain container deployment, networking, and realm exports.

Q3.2 CS Role: CS / Software Engineering members walk through Python OIDC verification logic and token refreshing.

Q3.3 Cyber Role: Cybersecurity members outline residual threat vectors (e.g., token theft, refresh token hijacking).

7. Clear Team Work Breakdown Structure (8 Members)

To ensure active participation of all group members, create a Role Allocation Matrix so that each member takes ownership of specific functions of the project to them in the team:

GROUP NAME:		**TEAM LEAD:**		
Role/Sub-Team	**Assigned Members**	**Student Numbers**	**Primary Course Domain Focus (Lab 4)**	**Comment**

8. SUBMISSION REQUIREMENTS

NOTE: After completing the task, save the work as **ICS341_LAB_4_GROUP#_TEAM LEAD STUDENT_NUMBER.pdf** and submit on Moodle on **Tuesday 27th October, 2026** not later than **23:59hrs** and **avoid** any form of **PLAGIARISM** instead where need be making citation to be academically ethical with **responsible AI** use.

ALL THE BEST!